THE JOURNEY OF A TINY MODEL (YOUR TLM) · EVERY NUMBER COMPUTED

Arithmetic Intelligence

User-friendly AI maths

Follow your TLM, a language model with 80 numbers: designed, started from random noise, trained on four sentences, fine-tuned, rewarded, wrapped in a chat harness. At every step, the same step in Llama 3.1 8B, and what it costs in compute, memory and GPUs.

EVERY CHAPTER HAS

- **Story bar**: which of the 5 stages we are in
- **Mental map**: the step highlighted
- **Comes in → Who does the work → Hands on**: how it connects
- **Our numbers**: computed, step by step
- **Python**: a short snippet you can run

PLUS

- **In a real model**: Llama 3.1 8B
- **Scale it up**: compute, memory, GPUs
- **Insight**, **Watch out**, **Myth vs reality**

THE ONLY MATHS YOU NEED

Multiply, add, and turn numbers into percentages. Where something harder appears (an exponent, a logarithm, a square root), the guide gives the result and one line on why it's there.

START HERE

Why you should (or shouldn't) care

Love maths or hate it, AI decisions now land on your desk. A few multiplications decide the bill, the speed and the failure modes.

You don't like maths

You won't have to do any. Every calculation is done and shown. What you get is the ability to **predict**: why a long chat slows down, why a bigger model costs more, why the same question gets two answers.

You like maths

The tiny model is complete and real. Every number can be checked by hand or with the Python snippets, and each chapter connects one operation to gigabytes, GPU hours and tokens per second.

Decisions this guide helps you make

Decision	Common guess	What the arithmetic says	Ch
Which model size?	Biggest is safest	Memory = parameters × bytes. 8B at 16-bit needs ≈ 16 GB; 70B needs ≈ 140 GB (several GPUs). Speed and cost scale with it.	17
Why is my long chat slow and expensive?	The provider is throttling	Every new token reads every earlier token's cached keys and values. Memory and work grow with conversation length.	15
Same question, different answers?	It's broken	The model outputs percentages; a word is then *sampled*. Lower temperature for consistency.	14
Trust a confident answer?	95% sure means right	A share is the model's preference among words, not a measured chance of being right. Our model gives "rain" 95% in a situation it never trained on.	14
Add our documents: fine-tune or paste in?	Fine-tune first	RAG puts relevant text into the prompt, where the model can read it; fine-tuning changes the weights. They solve different problems.	6
Which GPU to rent or buy?	Compare compute (TFLOPS)	Chat speed is capped by memory bandwidth and capacity far more than by compute.	18
Run it on a laptop?	4-bit is junk	4-bit ≈ ¼ of the 16-bit memory, usually with a modest quality loss.	17
Budget per word?	Price per word	Billing is per token. In English, 100 tokens ≈ 75 words; other languages often need more tokens per word.	2

ONE DECISION, WORKED THROUGH

A team wants every chat to include a 100-page manual (≈ 50,000 tokens) on Llama 3.1 8B.

```
Pasting it in:   50,000 tokens × 128 KB  ≈ 6.1 GB of cache per user
                 10 users at once        ≈ 61 GB  (about 4× the model)
Retrieving the 3 relevant pages (≈ 1,500 tokens):
                 1,500 × 128 KB          ≈ 0.18 GB per user
```

Same model, about 30× less memory per user. Chapters 15 and 17 show where the 128 KB comes from.

WHEN YOU CAN SKIP THIS

If you only chat casually, never choose models, never pay the bill and never act on AI output without checking, the myth boxes are enough. Everyone else: the scale tables are where the money is.

BEFORE WE START

Words you need

Every term used later, defined once, with its value in our tiny model and in Llama 3.1 8B.

Word	Plain meaning	Our tiny model	Llama 3.1 8B
Token	A piece of text (word or word-part) swapped for an id number	"love" = id 1	128,256 possible tokens
Vector (row)	A list of numbers. Each token is represented by one	4 numbers	4,096 numbers
Slot	**One position in a vector: the 1st number, the 2nd number… Also called a *dimension*. A slot has no name or meaning on its own; meaning lives in patterns across slots**	**4 slots**	**4,096 slots**
Embedding table	A lookup table with one row per token. The token id is the row number; the row is that token's starting vector	6 rows × 4 slots	128,256 rows × 4,096 slots
Matrix	A grid of numbers. Multiplying a vector by a matrix = many dot products at once	Detectors: 3 × 4	Feed-forward: 14,336 × 4,096 (×3)
Weight / parameter	One number set by training and saved in the model file. Same thing, two names	80	≈ 8.03 billion
Layer	One round of attention + feed-forward. Layers are stacked	1	32
Bytes per number	Storage per number. 16-bit = 2 bytes; 4-bit = ½ byte	80 × 2 = 160 bytes	≈ 16 GB at 16-bit
FLOP	One multiply or one add: the unit of compute	≈ 180 per token	≈ 16 billion per token
GPU memory / bandwidth	How much fits on the chip (GB) / how fast it can be read (TB/s)	—	H100: 80 GB / ≈ 3.35 TB/s
Training / inference	Setting the weights from data / using fixed weights to answer	Chapters 10–12 / 13–15	Weeks once / every request

THE FOUR COSTS TO WATCH IN EVERY CHAPTER

- **Compute** (FLOPs): how much multiplying. Drives training cost and prompt-reading speed.
- **Memory capacity** (GB): what must sit on the GPU at once. Decides which GPU, and how many users fit.
- **Memory bandwidth** (TB/s): how fast numbers can be read. Decides how fast the answer types out.
- **Number of GPUs**: needed when weights or cache don't fit on one, adding communication cost.

SETTING THE STAGE

What we are building, and how the story runs

We design a language model from nothing. No step is skipped and no number is invented.

THE GOAL

Goal	What it means	Done when
1. What the model must do	Continue text the way our training text does: after **"I love"**, say tea or coffee in the right proportion	tea ≈ 67%, coffee ≈ 33%
2. What training must do	Turn 80 random numbers into numbers that achieve goal 1	Loss close to its best possible value, 0.520
3. Your goal	See every number change on the way, and what each step costs at real scale	You can predict why a model is slow, costly or unsure

Start → finish: random model gives tea **18%** → after 500 training steps tea **66%**, coffee **33%**, rain **0.2%**.

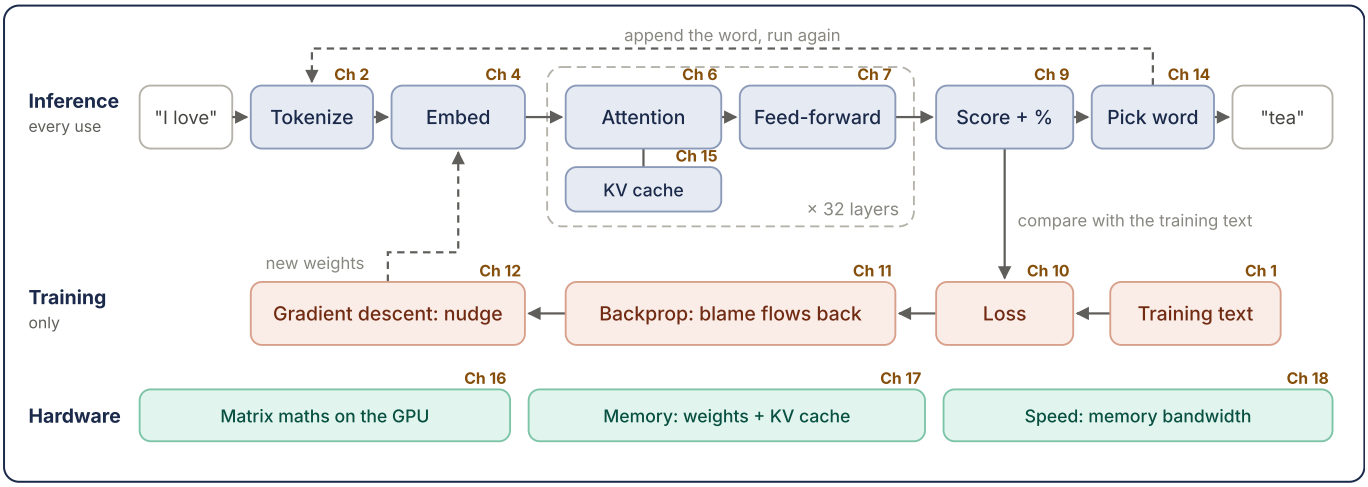
The story in six stages

Stage	Question	Who does the work	What comes out	Ch
1 Design	What should it learn, from what text, with what shape?	Us, the designers	Training text, vocabulary, the shape (80 empty slots for weights)	1–3
2 First run	What does a model with random weights do?	Embedding table, attention, feed-forward, scorer	A guess that is pure noise (tea 18%)	4–9
3 Train	How do random numbers become useful?	Loss, backprop, gradient descent	80 trained weights, saved and frozen	10–12
4 Use	What does the trained model do?	The same parts, now with trained weights, plus the picker and KV cache	"I love tea" (or coffee)	13–15
5 Scale	What changes at 8 billion weights?	GPU, memory, bandwidth	The bill: GB, GPUs, tokens per second	16–18
6 Beyond	How do today's assistants go further?	Fine-tuning, reinforcement learning, the harness (incl. chat interface)	A model that follows a preference, inside a product	19–21

Who does what: every part has one job

Part	Its one job	What it owns	Ch
Training text	Defines what "right" means	4 sentences	1
Tokenizer	Cuts text into pieces, swaps each for an id	A fixed word list (not learned)	2
Embedding table	Gives each id its starting row of numbers	24 weights	4
Attention	Lets a word pull information from earlier words	32 weights	6
KV cache	Keeps earlier words' keys and values so they aren't recomputed	Memory, no weights	15
Feed-forward	Adds information stored in its weights	24 weights	7
Scorer (softmax)	Turns the final row into a percentage per word	Reuses the embedding table	9
Picker	Chooses one word from the percentages	Settings: temperature, top-p	14
Loss	Measures how wrong the percentages were	Nothing; it compares	10
Backprop	Works out each weight's share of the blame	Nothing; it calculates	11
Gradient descent	Nudges every weight to reduce the loss	Setting: learning rate	12
GPU	Does all the multiplying; holds weights and cache in memory	Compute, memory, bandwidth	16–18
Fine-tuning data	Targeted text that shifts behaviour	Examples (here 3 sentences)	19
Reward	Scores answers the model produced	A rule, tests, or a reward model	20
Harness	Code around the model: chat history, prompts, tools, loops, checks	Ordinary software	21

The mental map: one diagram for the whole guide



HOW TO USE THE MAP

Every chapter shows this map in miniature with its step highlighted in amber, so you always know where you are. Inference is the top lane. Training runs the whole top lane, then adds the bottom lane. The amber Ch labels are chapter numbers.

What is real here, and what is simplified

Part	Our tiny model	Llama 3.1 8B	Same idea?
Embedding lookup, causal attention, shortcuts, softmax, cross-entropy loss	Yes, exactly as used in real models	Yes	✔ identical
Feed-forward switch	ReLU	SwiGLU (a gated, smoother switch)	✔ same purpose
Normalisation	None (numbers stay small enough)	RMSNorm before each sub-layer	✘ omitted
Word positions	None: order comes only from "look backwards only"	RoPE rotates Q and K by position	✘ omitted
Output scoring	Reuses the embedding table (as GPT-2 does)	A separate output table	✔ both are used in practice
Tokenizer	Whole words	128K sub-word pieces (BPE)	✔ same job
Starting weights	Random, spread 0.5, rounded to 1 decimal	Random, spread 0.02	✔ same idea
Optimiser	Plain gradient descent, 500 steps	AdamW, ≈ 15 trillion tokens	✔ AdamW is a refined version
Gradients	Computed by nudging each weight, checked against the chain rule (Ch 11)	Backprop via automatic differentiation	✔ same numbers

What this tiny model cannot show you

- **Capabilities at scale.** How large models generalise, follow instructions or reason. Four sentences can show the mechanism, not those abilities.
- **Post-training at scale.** Chapters 19–20 run fine-tuning and RL on the tiny model, but real instruction tuning and preference training use large curated datasets and human judgements.
- **Word order.** Without position encoding, our model knows order only through "look backwards only".
- **Where knowledge lives.** Our clean attention/feed-forward split is a mental model; in real models it's spread across many layers.
- **Real hardware speed.** GPU figures are idealised estimates, not benchmarks.

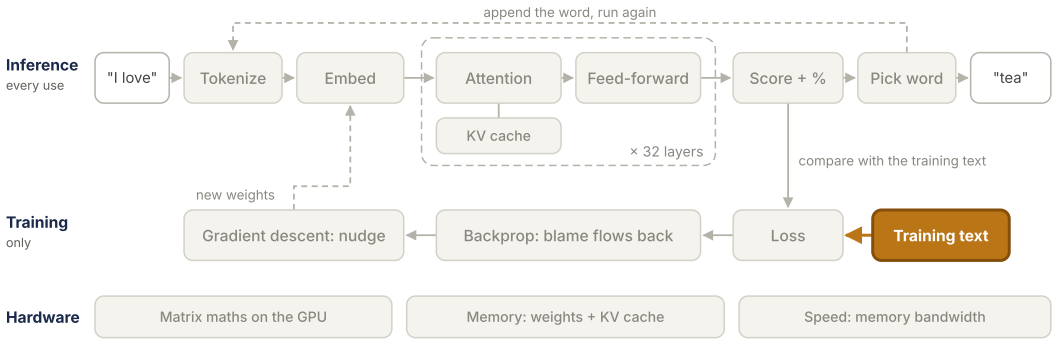
- **Understanding.** Similar rows show a useful statistical pattern, not evidence of understanding either way.

STAGE 1

Design

Before any maths runs, people decide three things: what the model should learn, from what text, and what shape it has.

STORY **1 Design** › 2 First run › 3 Train › 4 Use › 5 Scale › 6 Beyond



1 The goal and the training text

A language model learns one thing only: what usually comes next in its training text. So the text defines everything.

What do we want the model to learn, and from what?

COMES IN	WHO DOES THE WORK	HANDS ON
Our goal: continue "I love..."	Us, writing the training text	4 sentences → the tokenizer (Ch 2)

IN ONE LINE The training text defines what “right” means; pre-training learns its statistics.

We write four sentences:

```
I love tea
I love tea
I love coffee
I hate rain
```

The model learns to predict **each next word** from the words before it. That gives 8 prediction tasks:

Seen so far	Next words in the text	The best possible prediction
"I"	love, love, love, hate	love 75%, hate 25%
"I love"	tea, tea, coffee	**tea 67%, coffee 33%**
"I hate"	rain	rain 100%

- The right answer is **not** "always tea". It's a **split**, copied from how often each word follows in the text.
- "rain" only ever follows "hate", so after "love" it should get nearly 0%.

Count what follows what: this is the target — Python

```python
from collections import Counter
text = ["I love tea", "I love tea", "I love coffee", "I hate rain"]
after_love = Counter(s.split()[2] for s in text if s.startswith("I love"))
print(after_love)   # Counter({'tea': 2, 'coffee': 1})  ->  67% / 33%
```

IN A REAL MODEL (LLAMA 3.1 8B)

Llama 3.1 was pre-trained on **≈ 15 trillion tokens** of publicly available text (Meta model card). The task is identical: predict each next token. Everything it "knows" comes from patterns in that text.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Training text	4 sentences → ≈ 15 trillion tokens	Training compute ≈ 6 × parameters × tokens ≈ 7 × 10^{23} operations for 8B; Meta reports 1.46 million H100 GPU-hours	Text is streamed from storage, not held on the GPU	More text, more patterns learned	Cost grows with every token; data quality matters as much as size

INSIGHT

Data is one of the biggest factors in quality, alongside model size, architecture, training method and how the model is run. In our tiny case the link is direct: the text says tea 67%, so a perfect model says tea 67%. The objective rewards matching the text, not checking truth.

WATCH OUT

Bias and errors in the text become bias and errors in the model. Nothing in training checks whether the text is true.

MYTH

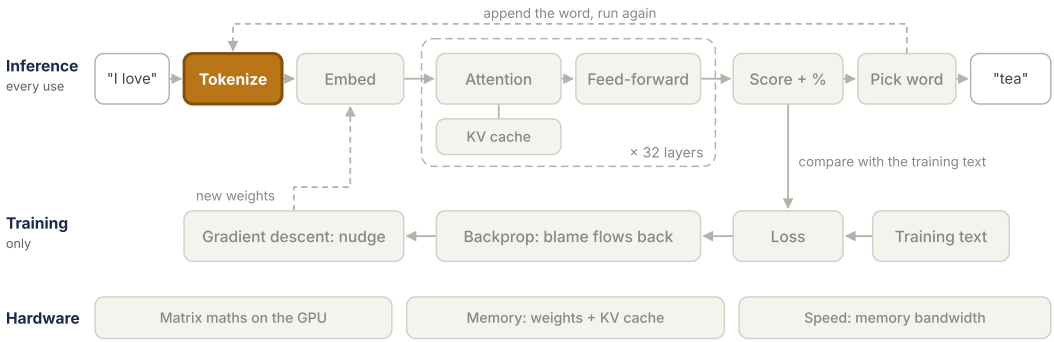
The model is taught facts and rules.

REALITY

Pre-training's only objective is predicting the next token. Large models trained this way on diverse text do pick up grammar, facts and reasoning patterns. The objective is simple; the capabilities that emerge from it at scale are not, and our 4-sentence model can't show them.

WATCH OUT

Chat assistants get a second phase after pre-training: instruction tuning and preference training (e.g. RLHF). Llama 3.1 8B *Instruct* is that post-trained version. Chapters 19–20 show both on our tiny model.



2 Tokenizer: building the vocabulary

A chip can only do arithmetic. The very first design job is a fixed list that turns text into numbers.

How does text become something a model can compute with?

COMES IN	WHO DOES THE WORK	HANDS ON
The training text (Ch 1) →	Tokenizer: a fixed list, built once	→ Token ids → the embedding table (Ch 4)

IN ONE LINE Models count tokens, not words, and so do prices and limits.

We list every distinct word in the text, in order of first appearance, and number them from 0. That list is the **vocabulary**; the numbers are **token ids**.

Token id	0	1	2	3	4	5
Word	I	love	tea	coffee	hate	rain

#	What happens	Our numbers
1	Text arrives	"I love"
2	Cut into pieces (tokens)	"I", "love"
3	Swap each for its id	0, 1

The tokenizer is **not learned** and holds no weights. It's built once before training and never changes.

Our tokenizer, and a real one — Python

```
words = "I love tea I love tea I love coffee I hate rain".split()
vocab = {w: i for i, w in enumerate(dict.fromkeys(words))}
print([vocab[w] for w in "I love".split()])   # [0, 1]

import tiktoken                                # pip install tiktoken
enc = tiktoken.get_encoding("cl100k_base")     # a real ~100K sub-word vocabulary
print(enc.encode("I love tea"))                # 3 ids, one per piece
```

IN A REAL MODEL (LLAMA 3.1 8B)

Real tokenizers split text into **sub-word pieces** (BPE), so rare words become several tokens. Llama 3's 128K vocabulary combines 100K tokens from tiktoken with 28K extra tokens for other languages, and averages ≈ 3.94 characters per token on English (Llama 3 paper).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Vocabulary	6 → 128,256	Final scoring is one dot product per vocabulary entry: 4,096 × 128,256 ≈ 525 M multiply-adds per token	Embedding table 525 M numbers; output table another 525 M (≈ 1 GB each at 16-bit)	Bigger vocabulary → fewer tokens per sentence → cheaper, faster, more text fits in context	Bigger tables; rare tokens learned poorly
Tokens per text	—	Every token is one more pass	Every token adds to the KV cache	English is cheap: 100 tokens ≈ 75 words	Languages that split into more pieces per word cost more for the same text

WATCH OUT

Prices, context limits ("128K") and speed are all in tokens, not words.

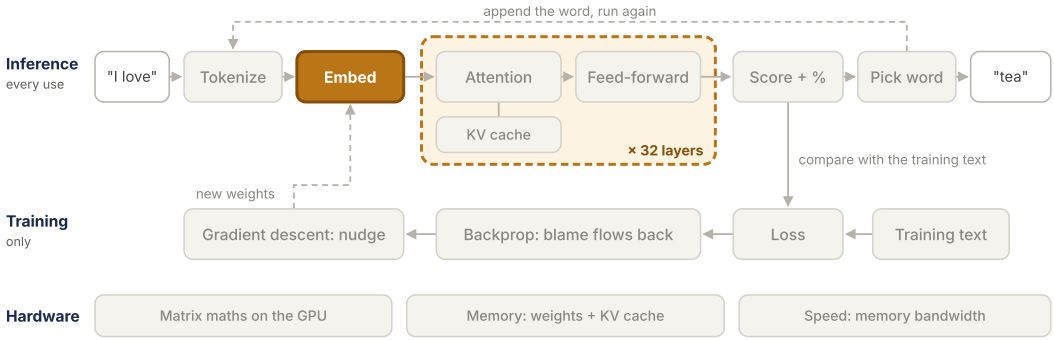
MYTH

The model reads letters like we do.

REALITY

It receives only ids. It never sees spelling, which is why counting letters in a word is surprisingly hard for it.

STORY **1 Design** › 2 First run › 3 Train › 4 Use › 5 Scale › 6 Beyond



3 The shape: slots, layers and our 80 weights

Before training, someone fixes the model's shape. Training can change the numbers but never the shape.

How big should each part be, and why did we pick these sizes?

COMES IN
The vocabulary: 6 tokens (Ch 2)

→ WHO DOES THE WORK
Us: choosing the sizes

→ HANDS ON
80 empty places for weights → filled at random (Ch 4)

IN ONE LINE The shape is fixed before training; the size knobs set the cost.

Choice	Ours	Why this number	Llama 3.1 8B	What we give up
Slots per token	4	The smallest size that does three jobs: • enough room for 6 words to settle into different positions • attention can shrink rows to a smaller size (4 → 2), as Llama does (4,096 → 128) • a dot product stays at 4 multiplications, doable by hand **2 slots:** too crowded, and the shrink can't be shown. **8 slots:** double the arithmetic; tables stop fitting the page.	4,096	Room to separate 128K tokens and their many meanings
Layers	1	One complete pass with every number shown. Llama's 32 layers repeat the same steps with different weights	32	No layer-by-layer refinement
Attention heads	1, size 2	One head shows the whole mechanism; size 2 makes Q, K, V visibly smaller than the token row	32 query heads × 128; 8 key/value heads	Can't look for several patterns at once
Feed-forward detectors	3	Enough to show detectors that fire and one that's switched off	14,336 per layer	Real feed-forward is 3.5× *wider* than the row; ours is narrower to keep tables short

Where the 80 weights live

Part	Shape	Count	Same part in Llama 3.1 8B	Ch
Embedding table	6 tokens × 4 slots	24	128,256 × 4,096 ≈ 525 M	4
Q, K, V matrices	3 × (4 → 2)	24	≈ 42 M per layer	6
Attention output	2 → 4	8		6
Detectors + write rows	3 × 4 + 3 × 4	24	≈ 176 M per layer	7
Total		**80**	**≈ 8.03 billion (incl. separate 525 M output table)**	

Count the weights, ours and Llama's — Python

```
ours = 6*4 + 3*(4*2) + 2*4 + 3*4 + 3*4            # = 80
d, L, ff, vocab, kv = 4096, 32, 14336, 128256, 1024
attn  = d*d + 2*(d*kv) + d*d                       # Q, K, V, output
ffn   = 3 * d * ff                                 # SwiGLU uses 3 matrices
llama = L*(attn + ffn) + 2*vocab*d                 # ~8.0e9 (norm weights add ~0.3M)
print(ours, f"{llama:,}")
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Width (slots)	4 → 4,096	Most matrices are width × width, so 2× width ≈ 4× compute per layer	≈ 4× the weights; KV cache 2×	More room to separate meanings	Cost grows with the square of width
Layers	1 → 32	∝ layers	∝ layers (weights and KV cache)	More rounds of combining information	Layers run one after another for each token: more layers, more delay

INSIGHT

"8B" is just this table filled in with big numbers. Every size knob in Llama is one we chose here, turned up.

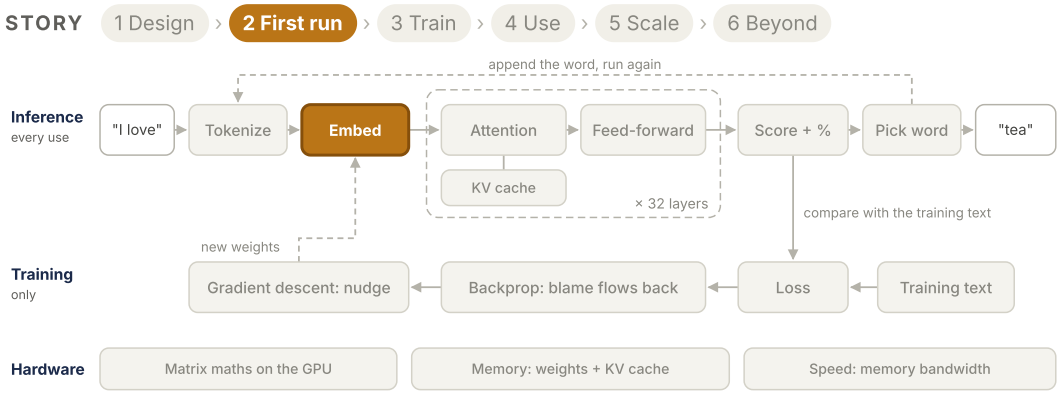
WATCH OUT

The shape is permanent. Changing slots or layers later means training a new model from scratch.

STAGE 2

First run: a model with random weights

We fill all 80 weights with random numbers and run "I love" through it. Every step works; none of it means anything yet. This is exactly where every real model starts.



4 Random birth and the embedding table

The newborn model is 80 random numbers. The first 24 form the embedding table.

What is the embedding table, and what's in it before training?

COMES IN
Token ids 0, 1 (Ch 2)

→

WHO DOES THE WORK
Embedding table: a lookup, no maths

→

HANDS ON
2 rows of 4 numbers → attention (Ch 6)

IN ONE LINE The embedding table is a learned lookup: one row per token.

- A **lookup table** with one row per token in the vocabulary.
- Each row is that token's **starting vector** (4 slots).
- The **token id is the row number**: "love" = id 1 = row 1.
- Using it is **pure lookup**: fetch the row. Its numbers are weights, so training will change them.

The newborn's embedding table (random, spread 0.5, rounded to 1 decimal)

Word	Token id (row)	Slot 1	Slot 2	Slot 3	Slot 4
I	**0**	**0.2**	**0.4**	**0.2**	**−0.7**
love	**1**	**0.5**	**0.2**	**−0.3**	**0.3**
tea	2	0.2	0.1	0.0	0.3
coffee	3	−0.4	−0.1	−0.2	0.3
hate	4	0.0	−0.1	−0.4	−0.1
rain	5	0.0	−0.1	0.6	0.5

Looking up "I love" fetches rows 0 and 1: `I [0.2, 0.4, 0.2, −0.7]`, `love [0.5, 0.2, −0.3, 0.3]`.

Random birth and lookup — Python

```python
import numpy as np
rng = np.random.default_rng(1)
E = rng.normal(0, 0.5, size=(6, 4)).round(1)   # the embedding table
ids = [0, 1]                                    # "I love"
X = E[ids]                                      # lookup = row indexing
```

IN A REAL MODEL (LLAMA 3.1 8B)

Llama 3.1 8B's table is 128,256 rows × 4,096 slots ≈ 525 million numbers (≈ 1 GB at 16-bit). It starts random too: its config sets the initial spread (initializer_range) to 0.02.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Rows × slots	6 × 4 → 128,256 × 4,096	Lookup costs almost no compute	≈ 525 M numbers ≈ 1 GB at 16-bit	Every token gets a rich starting point	Must sit in GPU memory even though each token uses one row

INSIGHT

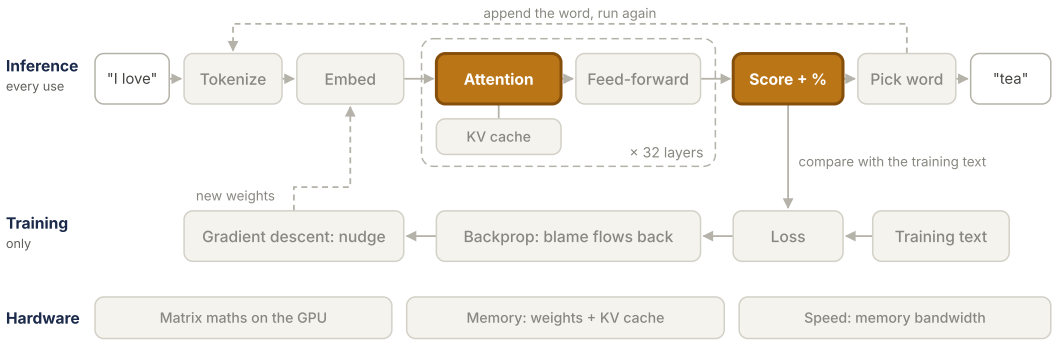
Slots have no names. Slot 3 doesn't mean anything; it's just position 3. Any meaning appears only after training (Ch 13).

MYTH

Someone designs the embedding table.

REALITY

It begins as random noise. Training shapes it.



5 Dot product: the one operation behind everything

Attention and scoring both rely on one question: how much do two rows agree?

How does the model put a number on "how related are these two rows?"

COMES IN	WHO DOES THE WORK	HANDS ON
Rows from the embedding table (Ch 4)	Dot product: multiply slot by slot, add up	A score → used by attention (Ch 6) and the scorer (Ch 9)

IN ONE LINE A dot product scores agreement, and most of the model is dot products.

Multiply two rows slot by slot, then add. One number comes out. Big positive = they point the same way; near zero = unrelated; negative = opposite.

NEWBORN NUMBERS: TEA VS COFFEE

```
tea    :  0.2   0.1   0.0   0.3
coffee : −0.4  −0.1  −0.2   0.3
product: −0.08 + −0.01 + 0.00 + 0.09  =  0.00
```

Newborn tea and coffee score **0.00**: unrelated, purely by chance. After training they score **1.23** (Ch 13). Nobody will tell the model they're similar; training will find it.

Dot product and cosine similarity — Python

```python
import numpy as np
def dot(a, b): return float(np.dot(a, b))          # multiply pairs, add
def cos(a, b): return dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
tea, coffee = np.array([0.2, 0.1, 0.0, 0.3]), np.array([-0.4, -0.1, -0.2, 0.3])
print(dot(tea, coffee), cos(tea, coffee))         # 0.0  0.0
```

IN A REAL MODEL (LLAMA 3.1 8B)

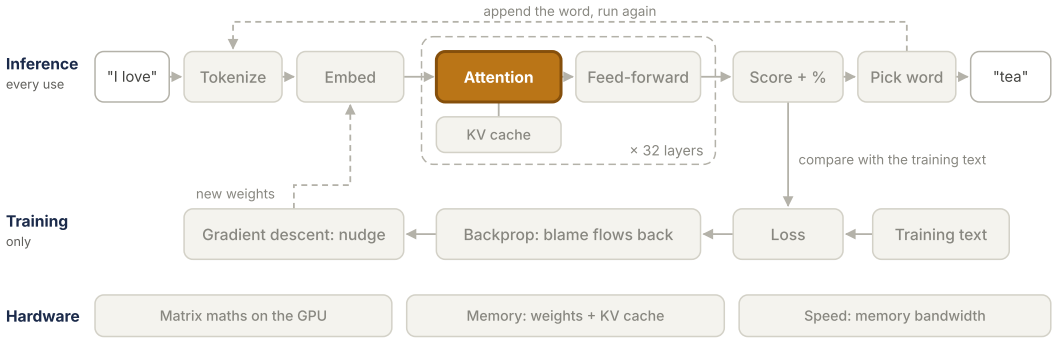
In every layer, each query is compared with every earlier key: one dot product of 128 numbers per head per earlier token. The final scoring is 128,256 dot products of 4,096 numbers, for every token.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Slots per dot product	4 → 4,096 (128 inside a head)	Cost ∝ slots	None extra	Richer comparisons	Tens of billions of them per token
Number precision	16-bit → 8-bit	GPUs run roughly 2× the 8-bit operations per second	½ the bytes	Faster and smaller	Small rounding errors

WATCH OUT

Length inflates the score. Dividing by both lengths gives **cosine similarity** (−1 to 1), which compares direction only.



6 Attention: a word pulls information from earlier words

This is the only step where words exchange information. Everything else handles each word alone.

How does "love" pull in information from "I"?

COMES IN	WHO DOES THE WORK	HANDS ON
Rows for "I" and "love" (Ch 4)	Attention: Q, K, V weights (32)	Updated "love" row → feed-forward (Ch 7)

IN ONE LINE Attention is the only step where tokens read each other.

Each token makes three smaller rows by multiplying its row by three weight matrices:

Row	Its job	"love" (newborn)	"I" (newborn)
Q query	Compared with every key: decides what this token pulls in	[−0.42, −0.70]	—
K key	What each token is matched on	[0.08, 0.83]	[−0.05, 0.09]
V value	What gets copied if matched	[−0.30, −0.34]	[−0.12, −0.05]

#	What happens	Our numbers
1	Q of "love" · each K, divided by √2 (√ of the head size) to keep scores in range	with I: −0.03 with love: −0.43
2	Softmax the scores into shares (Ch 9 explains softmax)	I 60%, love 40%
3	Blend the V rows by those shares	0.60 × V(I) + 0.40 × V(love) = [−0.19, −0.17]
4	An output matrix turns 2 numbers back into 4, which are added to love's row	[0.5, 0.2, −0.3, 0.3] + [−0.12, −0.03, −0.10, 0.16] = **[0.38, 0.17, −0.41, 0.46]**

Look backwards only: a token may only attend to itself and earlier tokens (the **causal mask**). "I" sees only itself (100%). This is what makes it a next-word predictor: it can't peek at the answer.

One attention head in NumPy — Python

```
def attention(X, Wq, Wk, Wv, Wo):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    s = Q @ K.T / np.sqrt(K.shape[1])                # every query vs every key
    s += np.triu(np.full(s.shape, -np.inf), 1)       # causal mask: no peeking ahead
    w = np.exp(s - s.max(1, keepdims=True)); w /= w.sum(1, keepdims=True)
    return (w @ V) @ Wo                              # blend values, back to 4 slots
```

IN A REAL MODEL (LLAMA 3.1 8B)

32 query heads per layer, each with rows of 128 numbers. Keys and values use only 8 heads, each shared by 4 query heads (grouped-query attention), which shrinks the cache 4×. Llama also rotates Q and K by position (RoPE), which our model omits.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Context length	2 → up to 131,072 tokens	Every token scores every earlier one: 16× more tokens ≈ 256× more score work	KV cache grows in step with tokens (128 KB each)	Model can use more of your document	Long prompts are slow and pricey
K/V heads	1 → 8 (shared by 32 query heads)	Barely changes	4× smaller cache than 32 separate K/V heads	More users and longer context per GPU	A trade-off chosen at training time

INSIGHT

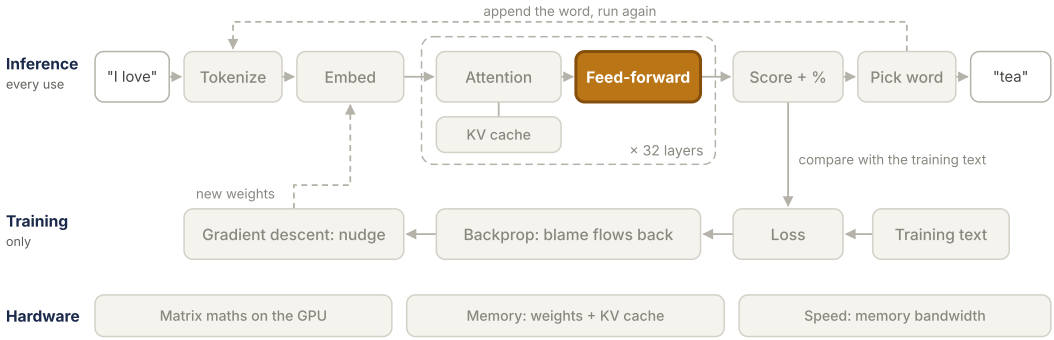
Attention is the only place where one token reads another, so text in your prompt influences the answer through attention. That's the in-model half of RAG. RAG as a system has three steps: **retrieve** relevant passages, **insert** them into the prompt, **generate**. Inserting text makes it available; it doesn't guarantee the model uses it correctly.

MYTH

Attention means the model focuses like a person.

REALITY

It's a weighted average: here 60% of one value row and 40% of another, blended and added.



7 Feed-forward: detectors, a switch, and a write

Attention mixes words. Feed-forward adds information from the weights themselves, one word at a time.

What happens to "love" after attention?

COMES IN	WHO DOES THE WORK	HANDS ON
Updated "love" row (Ch 6)	Feed-forward: 3 detectors (24 weights)	A change to add to "love" → shortcut (Ch 8)

IN ONE LINE Feed-forward transforms each token on its own, using stored weights.

#	What happens	Our numbers
1	**Detect**: each detector takes a dot product of the row with its own weights	[0.38, 0.17, −0.41, 0.46] → scores [0.34, −0.06, 0.15]
2	**Switch** (ReLU): negative → 0, positive unchanged	[0.34, 0.00, 0.15]: detector 2 is off
3	**Write**: each detector adds its own stored row, scaled by how strongly it fired	change = [−0.30, −0.07, 0.19, −0.06]

A single detector is what textbooks call a **neuron**: a row of weights and one dot product.

Why the switch matters: without it, "detect" and "write" collapse into one fixed table, so stacking 32 layers would be no better than one. With it, *which* detectors fire depends on the input. Chapter 13 measures the difference in the trained model.

Feed-forward in NumPy — Python

```
def feed_forward(H, W1, W2):
    scores = H @ W1.T                # detect: one dot product per detector
    on = np.maximum(scores, 0)       # switch: ReLU
    return on @ W2                   # write: scaled sum of write rows
```

IN A REAL MODEL (LLAMA 3.1 8B)
14,336 detectors per layer, 32 layers. Llama's switch is SwiGLU: a second set of 14,336 weights acts as a gate, so it uses 3 matrices instead of 2.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

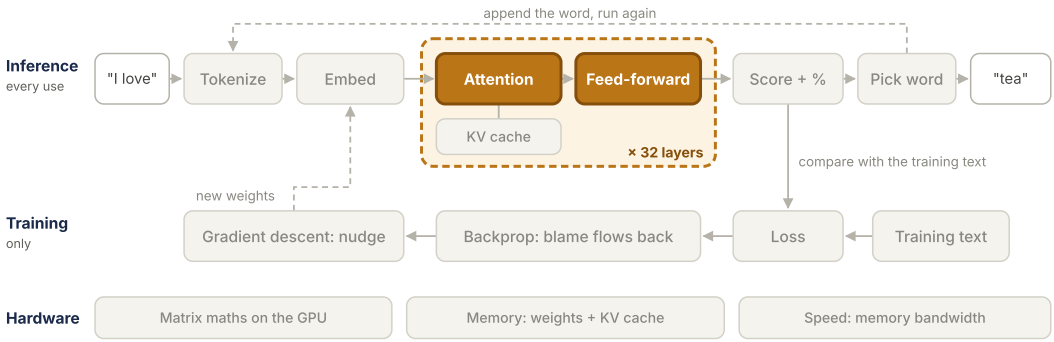
Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Detectors	3 → 14,336 per layer	Every token runs every detector: ≈ 80% of each layer's work	≈ 80% of the weights (≈ 176 M of ≈ 218 M per layer)	Room for many stored patterns	Every token pays for all of them
Switch type	ReLU → SwiGLU	+50% feed-forward compute	+50% feed-forward weights	Better quality per weight (found by experiment)	An extra matrix

INSIGHT

A useful mental model: attention **moves** information between tokens; feed-forward **transforms** each token using its weights. Research finds much factual recall involves feed-forward layers, but knowledge and computation are spread across both, and across many layers. Treat the split as a guide, not a complete map.

WATCH OUT

Real detectors rarely mean one clean idea. Each responds to a mix of patterns, and each idea is spread over many detectors (superposition).



8 One layer: add, don't replace

Attention and feed-forward each compute a change. The layer adds both changes to the original row.

How do the pieces combine inside one layer?

COMES IN
Changes from attention (Ch 6) and feed-forward (Ch 7)

→

WHO DOES THE WORK
Shortcut connections: plain addition

→

HANDS ON
Final "love" row → scorer (Ch 9)

IN ONE LINE Each layer adds changes; it never replaces the row.

Stage	Operation	"love" row (newborn)
Start	Embedding lookup	`[0.50, 0.20, −0.30, 0.30]`
Attention	Change from earlier tokens	`+[−0.12, −0.03, −0.10, 0.16]`
Shortcut	Add	`=[0.38, 0.17, −0.41, 0.46]`
Feed-forward	Detect, switch, write	`+[−0.30, −0.07, 0.19, −0.06]`
Shortcut	Add	**`=[0.08, 0.10, −0.22, 0.40]`**

These added paths are called **residual shortcuts**. The original word is never overwritten, only adjusted. Without them, 32 layers of rewriting would wash out the original token.

One full layer — Python

```
H  = X + attention(X, Wq, Wk, Wv, Wo)     # add attention's change
H2 = H + feed_forward(H, W1, W2)          # add feed-forward's change
```

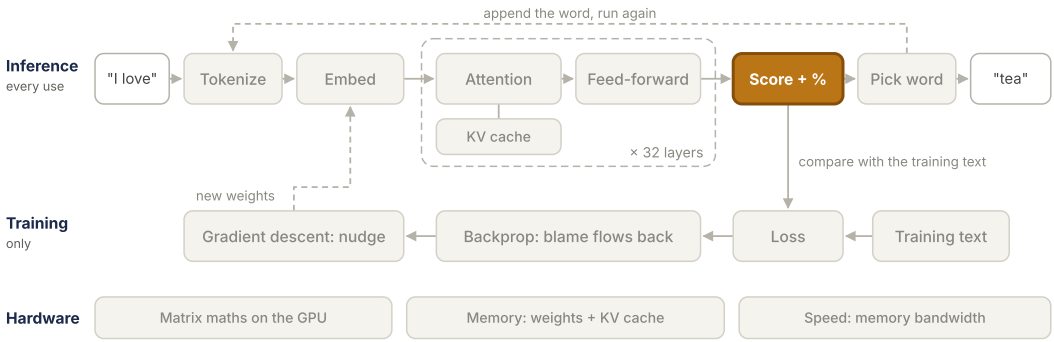
IN A REAL MODEL (LLAMA 3.1 8B)

32 layers in a row, same steps, different weights. Before each sub-step Llama rescales the row to a standard size (RMSNorm) to keep numbers stable over 32 layers; our model omits it.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Layers	1 → 32	∝ layers	∝ layers	More rounds of combining	Each token waits for all 32 in turn

MYTH	REALITY
Each layer replaces the word with something new.	Each layer adds a correction to a running total. After 32 layers the row is the original embedding plus 64 additions.



9 Scores to percentages: the newborn's guess

The final row now votes for each word in the vocabulary. For a random model, the vote is noise.

How does a row of numbers become "x% tea"?

COMES IN
Final "love" row (Ch 8)

→ WHO DOES THE WORK
Scorer: dot products + softmax

→ HANDS ON
Percentages → loss (Ch 10) or picker (Ch 14)

IN ONE LINE The output is a percentage for every token, not a word.

#	What happens	Our numbers
1	**Score** each word: dot product of the final row with that word's embedding row. Raw scores are called **logits**	I −0.26, love 0.24, tea 0.14, coffee 0.12, hate 0.04, rain 0.06
2	**Softmax**: raise e (≈ 2.72) to each score, then divide by the total so all shares add to 100%	see table

	I	love	tea	coffee	hate	rain
Newborn share	12.0%	19.8%	18.0%	17.6%	16.1%	16.5%

Every word gets 12–20%. Tea is at **18%**; the target is 67%. **Stage 2 ends here:** every part works, but the weights are random, so the answer is noise. Stage 3 fixes the weights.

Why the exponent?

- Scores can be negative; percentages can't. e raised to anything is positive.
- It widens gaps, so a clear winner stands out (the trained model will show this).

Softmax — Python

```python
def softmax(z):
    e = np.exp(z - z.max())      # subtracting the max avoids overflow; same result
    return e / e.sum()
```

IN A REAL MODEL (LLAMA 3.1 8B)
The final row has 4,096 numbers and is scored against 128,256 rows, giving 128,256 percentages per token.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Vocabulary scored	6 → 128,256 per token	≈ 525 M multiply-adds per token	Small for one user; in training, scores for millions of tokens at once need careful chunking	The model can say any of 128K tokens	Big output step

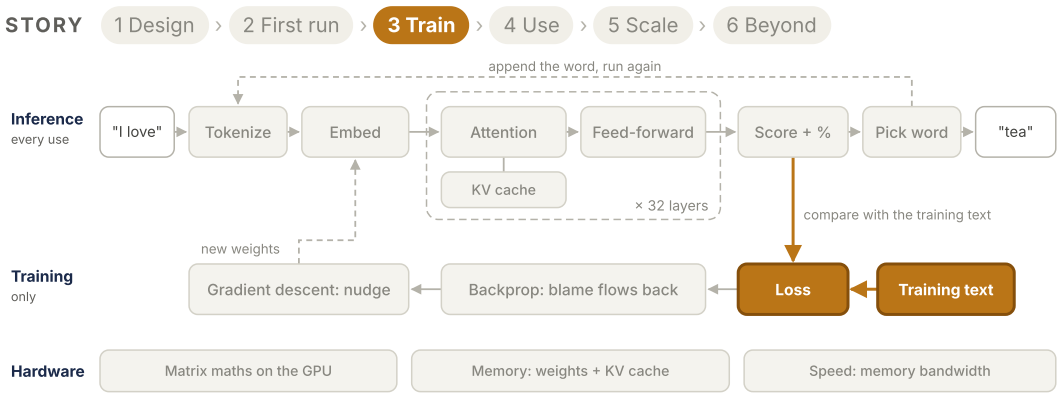
WATCH OUT

A share is a **preference**, not a probability of being correct. A model can put 95% on a wrong answer (Ch 14).

STAGE 3

Train: turning noise into knowledge

Training repeats one loop: run the model, measure the mistake, find who is to blame, nudge every weight. Our run: 500 loops.



10 Loss: one number for "how wrong"

To improve, the model needs a single number to push down. Everything in training serves that number.

How do we score percentages against the training text?

COMES IN
Percentages (Ch 9) + the training text (Ch 1)

→ WHO DOES THE WORK
Loss: cross-entropy

→ HANDS ON
One number → backprop (Ch 11)

IN ONE LINE Loss measures surprise; training pushes it down.

The loss used for language models is **cross-entropy**: **how surprised was the model by the word that actually came next?** It's −ln(share given to the actual next word). The model is scored on all 8 predictions from Chapter 1 and the results averaged.

Share given to the actual next word	Loss	Meaning
100%	0	No surprise
67%	0.40	Mild surprise
18.0% (newborn, "I love" → tea)	**1.72**	**Very surprised**
1%	4.61	Confidently wrong

Average over all 8 predictions	Loss
Newborn model	1.864
Guessing evenly among 6 words	1.792
Best possible for this text (the 75/25 and 67/33 splits)	**0.520**

The best possible loss is **not 0**: after "I love" the text itself is uncertain (tea or coffee), so even a perfect model is a little surprised.

Cross-entropy loss — Python

```
p = softmax(logits)          # the model's shares
loss = -np.log(p[target])    # target = id of the actual next word
```

IN A REAL MODEL (LLAMA 3.1 8B)

A model guessing evenly among 128,256 tokens would score ln(128,256) ≈ 11.8. Training on ≈ 15 trillion tokens pulls the average loss far below that.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Predictions scored	8 → trillions	Each token in the training text is one prediction	Text is streamed	More predictions, more learned	Training cost ∝ tokens

INSIGHT

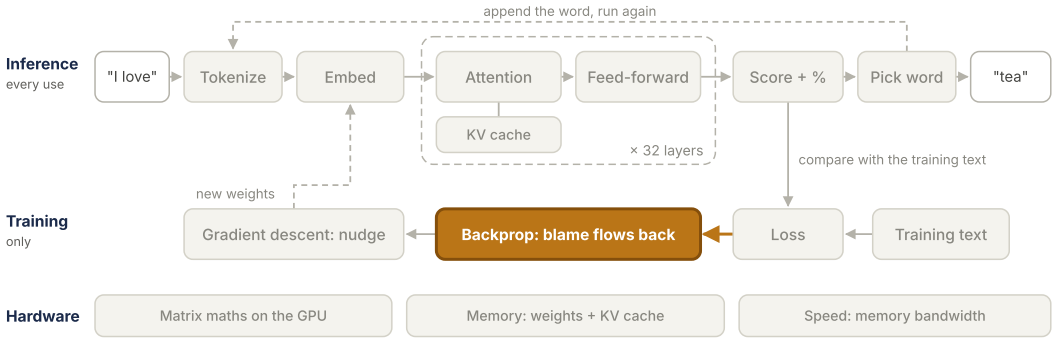
Confident mistakes cost most. 1% on the right word costs 4.61, while 67% costs only 0.40. Training fixes confident errors first.

MYTH

Low loss means the model is right.

REALITY

Low loss means it predicts its training text well. That is related to, but not the same as, giving correct or useful answers on new questions.



11 Backprop: finding each weight's share of the blame

80 weights contributed to the mistake. Training needs to know, for each one, which way to move.

How do we know whether to increase or decrease one particular weight?

COMES IN	WHO DOES THE WORK	HANDS ON
The loss (Ch 10) →	Backprop: the chain rule, run backwards	→ A gradient for every weight → gradient descent (Ch 12)

IN ONE LINE Backprop gives every weight a direction to move.

The **gradient** of a weight = how much the loss changes if that weight rises a tiny bit. **Backprop** computes every gradient by working backwards from the loss, multiplying effects along the way (the **chain rule**).

Example: detector 1's write into slot 3 (currently 0.6), for the single prediction "I love" → tea.

#	What happens	Our numbers
1	**Blame on each word's score** = its share − (1 if it's the actual next word, else 0). This simple rule comes out of softmax + cross-entropy	tea 18.0% − 1 = **−0.82**; every other word = its own share, e.g. love +0.20
2	**Blame on slot 3 of the final row**: every score used slot 3, so add up (word's blame × that word's slot 3)	= **−0.036**
3	**Blame on the weight**: its effect on slot 3 was scaled by detector 1's firing	0.339 × −0.036 = **−0.0123**

Check: we also nudged the weight by 0.00001, re-ran the model and measured the change in loss: **−0.0123**. Same number.

A negative gradient means "raise this weight and the loss falls".

Check a gradient by nudging (how we verified it) — Python

```
def numeric_grad(loss_fn, W, i, j, eps=1e-5):
    old = W[i, j]
    W[i, j] = old + eps; up = loss_fn()
    W[i, j] = old - eps; down = loss_fn()
    W[i, j] = old
    return (up - down) / (2 * eps)
# PyTorch does backprop for you:  loss.backward(); W.grad
```

IN A REAL MODEL (LLAMA 3.1 8B)

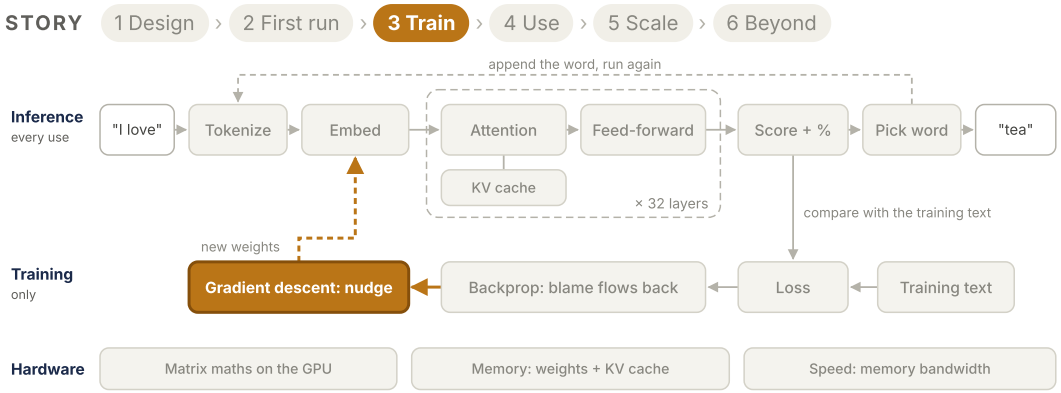
One backward pass computes gradients for all ≈ 8 billion weights at about twice the cost of the forward pass, so a training step costs ≈ 3× a forward pass (the source of the 6 × parameters × tokens rule).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Backward pass	—	≈ 2× forward	Intermediate values from the forward pass must be kept: grows with batch × context × layers	Exact gradients for every weight in one sweep	Memory-heavy; a common fix recomputes some values instead of storing them

INSIGHT

Look at step 1: blame is negative only for the actual next word (push it up) and positive for every other word (push down, in proportion to the share it took).



12 Gradient descent: the training run

Now we repeat the loop 500 times and watch random numbers become a model.

How do the weights change, and what do we get at the end?

COMES IN	WHO DOES THE WORK	HANDS ON
Gradients for all 80 weights (Ch 11) →	Gradient descent: weight − learning rate × gradient	→ 80 trained weights, saved and frozen → use (Ch 13)

IN ONE LINE Training is measure → blame → nudge, repeated.

Update rule: new weight = old weight − **learning rate** × gradient. Ours: learning rate 0.1, all 8 predictions per step, 500 steps.

Step	Loss	"I love" → tea	→ coffee	→ rain	"I" → love / hate	tea–coffee similarity
0	1.864	18%	18%	16.5%	14% / 15%	0.00
10	1.745	19%	18%	16.0%	18% / 16%	0.20
25	1.641	22%	19%	14.4%	21% / 14%	0.40
50	1.356	37%	22%	8.4%	32% / 9%	0.61
100	0.862	60%	26%	8.3%	70% / 6%	0.81
150	0.630	63%	30%	4.4%	75% / 14%	0.88
200	0.559	65%	31%	1.1%	80% / 18%	0.90
300	0.531	66%	33%	0.8%	70% / 29%	0.91
500	**0.523**	**66%**	**33%**	**0.2%**	**73% / 26%**	**0.92**
Target	*0.520*	*67%*	*33%*	*0%*	*75% / 25%*	—

- The loss falls from 1.864 to 0.523, almost the best possible (0.520).
- The model learned the **split**, not just "tea": 66% / 33%.
- **Nobody told it tea and coffee belong together.** Their cosine similarity went from 0.00 to 0.92, because they appear in the same place in the text. That's a useful statistical pattern, not proof of understanding.

Learning rate: too small, right, too large

Learning rate	Loss at step 0	Step 10	Step 50	Step 100
0.01	1.864	1.844	1.789	1.746 (barely moving)
0.1	**1.864**	**1.745**	**1.356**	**0.862**
1.0	1.864	2.245 (worse)	blew up: the numbers overflowed	

The whole training loop — Python

```
for step in range(500):
    grads = gradients(weights, text)       # backprop (Ch 11)
    for name in weights:
        weights[name] -= 0.1 * grads[name] # nudge every weight downhill
# then save: the weights are frozen for use
```

IN A REAL MODEL (LLAMA 3.1 8B)

Llama 3.1 used AdamW (a refined gradient descent), a peak learning rate of 3×10^{-4} for 8B, ≈ 15 trillion tokens, and 1.46 million H100 GPU-hours (Meta model card and Llama 3 paper).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Training memory per weight	Using: 2 bytes → training: ≈ 16 bytes	Each step: forward + backward + update	Mixed-precision Adam: 2 (weight) + 2 (gradient) + 12 (fp32 copy + 2 running averages) = 16 bytes × 8 B ≈ 128 GB, before intermediate values	Stable, fast learning	Training 8B needs several 80 GB GPUs; using it needs one
Batch size	8 predictions → millions of tokens per step	Big matrices keep the GPU busy	More intermediate values held	Steadier gradients, higher throughput	Memory

INSIGHT

No step in the loop is labelled “understand”. Training is this rule repeated: measure, find blame, nudge. Useful structure, like tea and coffee getting similar rows, comes out of it. Whether large models trained this way “understand” is debated; the mechanism itself is not.

WATCH OUT

Learning rate is a real risk. Too small wastes compute; too large makes training blow up, and large runs watch for exactly this.

MYTH

Training stores the text in the model.

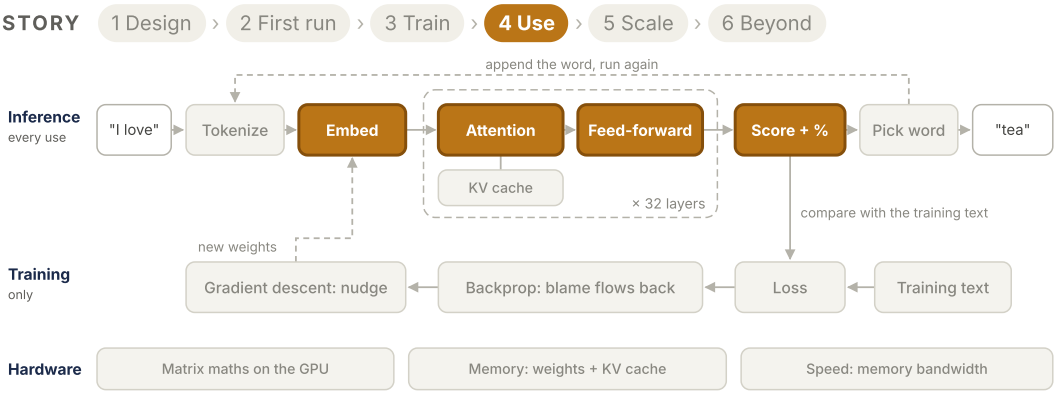
REALITY

Training adjusts 80 numbers. Our model holds no sentences, only numbers that reproduce the 67/33 pattern.

STAGE 4

Use: the trained model at work

Same steps as Stage 2, same arithmetic. Only the 80 numbers are different. This is inference: weights are read, never changed.



13 The trained model answers "I love"

Run exactly the same pipeline as Chapter 4–9, with trained weights.

What changed inside, step by step?

COMES IN
Trained weights (Ch 12) + "I love"

→

WHO DOES THE WORK
Every inference part, now trained

→

HANDS ON
Percentages → picker (Ch 14)

IN ONE LINE Same code, different numbers: a trained model is a file of weights.

Step	Newborn	Trained	Ch
Embedding row for "love"	`[0.5, 0.2, −0.3, 0.3]`	`[0.91, 1.44, −0.77, 0.70]`	4
Attention shares (I / love)	60% / 40%	60% / 40%	6
Row after attention	`[0.38, 0.17, −0.41, 0.46]`	`[2.13, 2.72, −4.57, 3.93]`	6
Detector scores	`[0.34, −0.06, 0.15]`	`[2.35, −0.74, 3.20]`	7
Final row	`[0.08, 0.10, −0.22, 0.40]`	`[−3.26, −1.72, −2.41, 5.01]`	8
Scores: tea / coffee / rain	0.14 / 0.12 / 0.06	7.81 / 7.12 / 2.13	9
Shares: tea / coffee / rain	**18% / 18% / 16%**	**66% / 33% / 0.2%**	**9**

A useful pattern appeared in the embedding table

Pair	Newborn cosine	Trained cosine
tea – coffee	0.00	**0.92**
tea – rain	0.48	0.22
love – hate	0.24	0.57

Tea and coffee became neighbours because both follow "I love". Slots still have no names; the pattern is in their *relative* positions. This is a learned statistical representation: useful for prediction, but cosine similarity alone isn't evidence of understanding.

The switch, measured

Trained detector scores [2.35, −0.74, 3.20]: detector 2 is negative and is switched off. With the switch, the write is [−5.40, −4.44, 2.15, 1.08]; with the switch removed, it would be [−5.15, −4.74, 2.10, 0.74]. Different input words switch on different detectors; that input-dependence is what makes stacked layers useful.

Use the trained model — Python

```
X  = E[[0, 1]]                          # "I love"
H  = X + attention(X, Wq, Wk, Wv, Wo)
H2 = H + feed_forward(H, W1, W2)
p  = softmax(H2[-1] @ E.T)              # tea 0.66, coffee 0.33
```

INSIGHT

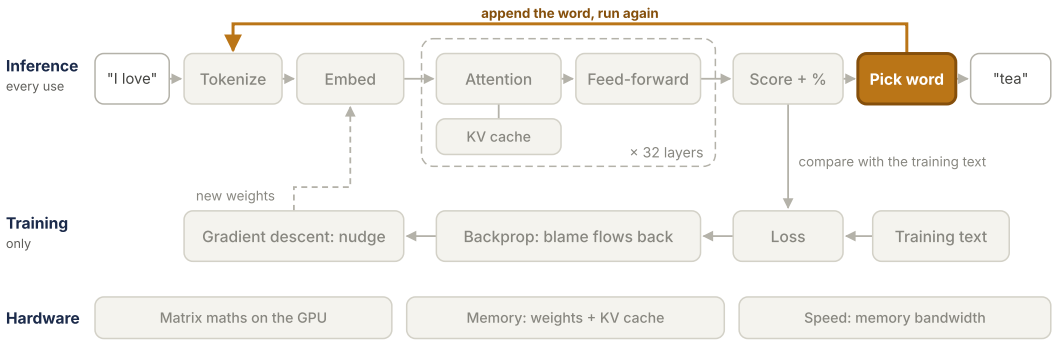
Nothing new was added between Stage 2 and Stage 4. Same code, same arithmetic, different 80 numbers. A trained model is just a file of numbers.

MYTH

The model looks up the answer.

REALITY

It computes scores for every word and ranks them. "Knowing" means the right words score highest.



14 Picking the word, then going again

The model outputs percentages. Something else chooses a word, appends it, and runs the model again.

How does a list of percentages become text, and why can answers differ?

COMES IN
Percentages (Ch 13)

→

WHO DOES THE WORK
Picker: sampling with temperature / top-p

→

HANDS ON
One word, appended → next step (Ch 15)

IN ONE LINE The picker adds randomness, and a share isn't a measure of reliability.

Temperature: divide the scores before softmax

Temperature	tea	coffee	rain	Behaviour
0.5	80%	20%	0.0%	Sharper, more repetitive
1	**66%**	**33%**	**0.2%**	**As trained**
2	55%	39%	3.2%	Flatter, riskier

Method	Rule	Our result
Greedy	Always the top word	tea, every time
Sampling	Weighted random pick	tea about 2 times in 3, coffee about 1 in 3
Top-p 0.9	Keep the top words until they cover 90%, sample among them	tea + coffee ≈ 100% → rain is never picked

The loop, and a warning

We append "tea" and run again on "I love tea". Our training text **never continued after "tea"**, so nothing was learned there. The model still answers, and confidently: **rain 95%**.

WATCH OUT

A model always produces percentages, even where it learned nothing. Here a 95% share comes from a situation the training text never covered. That's one mechanism behind confident errors, not a full explanation of hallucination in large models. Keep three things apart: a **token share** (the model's preference), **confidence in a whole answer**, and **measured reliability** (how often answers like this are right). They're related, but not interchangeable. Real models also learn an end-of-text token, so they know when to stop.

Sampling, ours and with Hugging Face transformers — Python

```
rng = np.random.default_rng()
next_id = rng.choice(len(p), p=softmax(logits / 0.7))   # temperature 0.7

from transformers import AutoTokenizer, AutoModelForCausalLM
name = "meta-llama/Llama-3.1-8B"                        # gated: accept licence first
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name, torch_dtype="auto", device_map="auto")
ids = tok("I love", return_tensors="pt").to(model.device)
out = model.generate(**ids, max_new_tokens=20, do_sample=True, temperature=0.7, top_p=0.9)
print(tok.decode(out[0]))
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

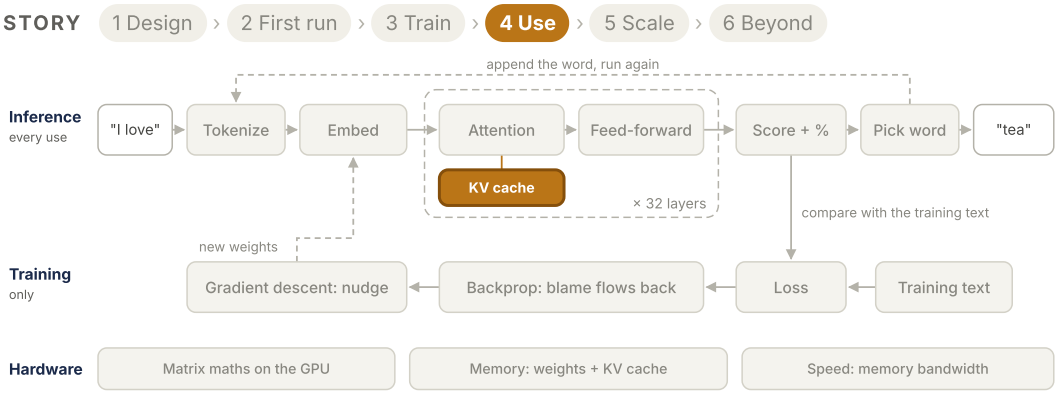
Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Output length	1 word → hundreds or thousands	Each output token is one full pass, one after another	Each adds 128 KB of cache	Longer, fuller answers	Output tokens are slow; APIs typically price them higher than input
Hidden reasoning tokens	—	Reasoning models may write thousands of tokens before answering	Same cache growth	Better on hard problems	You pay for tokens you may never see

MYTH

Temperature 0 makes the model accurate.

REALITY

It makes it consistent. If its top choice is wrong, it's wrong every time.



15 KV cache: keep the keys and values

To handle "I love tea", attention needs keys and values for all three words. Two of them were already computed last step.

What is saved between steps, and does it change the answer?

COMES IN	WHO DOES THE WORK	HANDS ON
New token "tea" (Ch 14)	KV cache: memory, no weights	Cache [I, love, tea] → next step

IN ONE LINE The cache saves computing, not reading or memory.

Because tokens only look backwards, earlier tokens' K and V rows **never change** when new tokens arrive. So they're computed once and stored.

TRAINED MODEL, STEP 3: NEW TOKEN "TEA"

```
Compute fresh for "tea" only:  Q = [1.79, −0.31]  K = [−0.06, −0.47]  V = [0.16, 0.61]

token   K               V                status
I       [0.21, 1.64]     [−1.13, −2.82]   reused
love    [0.48, 1.90]     [−1.42, −2.52]   reused
tea     [−0.06, −0.47]   [0.16, 0.61]     added now

Shares for tea's query:  I 29%   love 38%   tea 33%
```

Check: recomputing everything from scratch gives the same shares to every decimal (yes). The cache changes the cost, never the answer.

	Kept?	Why
K and V of earlier tokens	✔	Never change; every future query needs them
Q of earlier tokens	✘	Only used once, when that token was newest
Weights	n/a	Read again in full every step (Ch 18)

KV cache with transformers (generate uses it automatically) — Python

```python
out = model(**ids, use_cache=True)
past = out.past_key_values                        # cached K and V, every layer
next_id = out.logits[:, -1].argmax(-1, keepdim=True)
out = model(input_ids=next_id, past_key_values=past, use_cache=True)  # only 1 new token
```

IN A REAL MODEL (LLAMA 3.1 8B)

Per token: 2 (K and V) × 32 layers × 8 K/V heads × 128 numbers × 2 bytes = 131,072 bytes = **128 KB**. 8,192 tokens = 1 GB; the full 131,072-token context = **16 GB**, as much as the weights.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Cached tokens × users	3 rows → 128 KB × tokens × users	Saves recomputing K and V for every earlier token	1 user × 131K tokens = 16 GB; 10 users × 32K = 40 GB	Each step computes only the new token	The cache, not the weights, limits how many users fit

INSIGHT

The cache removes repeated **computing**, not repeated **reading**. The new query still compares with every stored key.

MYTH

The KV cache stores the conversation text.

REALITY

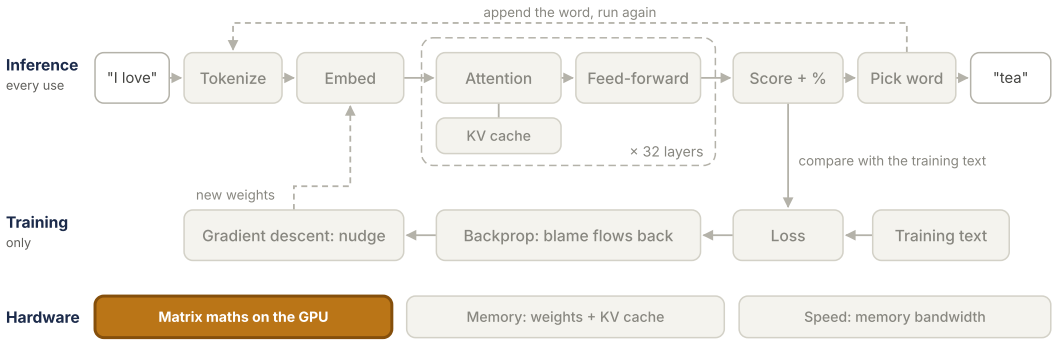
It stores K and V numbers per layer, per head, per token. The text itself is just a short list of ids.

STAGE 5

Scale: same maths, 100 million times bigger

Our 80 weights fit in 160 bytes. Llama 3.1 8B needs a data-centre GPU. Here is where the cost comes from.

STORY 1 Design › 2 First run › 3 Train › 4 Use › **5 Scale** › 6 Beyond



16 Matrices and GPUs

Every step in this guide was a batch of dot products. A GPU's job is doing thousands of them at once.

Why does AI run on GPUs?

COMES IN	WHO DOES THE WORK	HANDS ON
All the dot products (Ch 5–13) →	GPU: thousands of multiply-add units	→ Speed limits → memory (Ch 17)

IN ONE LINE GPUs win because the dot products are independent.

A **matrix multiplication** does many independent dot products at once. Both tokens through all 3 detectors = 6 dot products in one operation. A GPU runs thousands of such operations in parallel.

	Our model	Llama 3.1 8B
Operations per token (≈ 2 × weights)	≈ 180	≈ 16 billion
Reading a 1,000-token prompt	≈ 4 million (attention dominates in a model this small)	≈ 16 trillion (attention adds only a few %)
H100 peak (dense 16-bit)	≈ 989 trillion operations per second	

Many dot products, one call — Python

```
scores = H @ W1.T     # (tokens x 4) @ (4 x detectors): every pair at once
# On a GPU with PyTorch the same line runs on thousands of cores:
# scores = H.cuda() @ W1.cuda().T
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

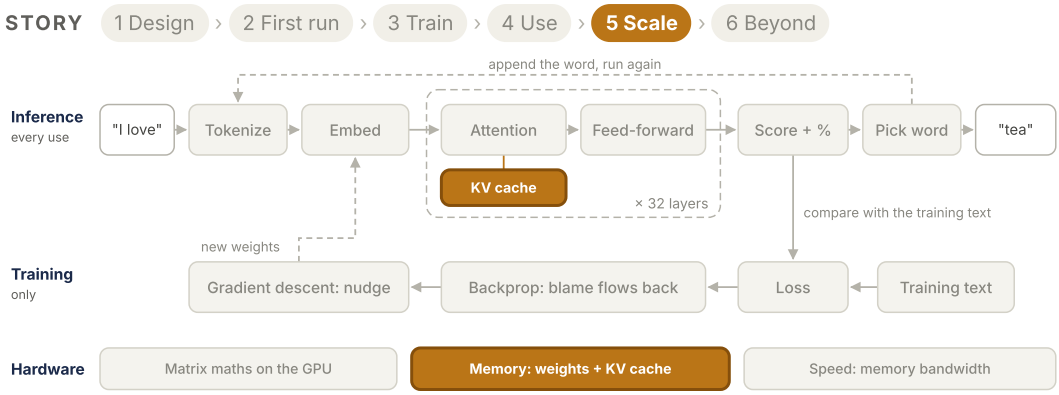
Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Rows per matrix (tokens or users together)	2 → thousands	Big matrices keep the GPU busy	More intermediate values	High throughput	Each user may wait longer
Model bigger than one GPU	—	Split across GPUs that exchange results every layer	Spread across GPUs	Run models that don't fit one chip	Needs fast GPU-to-GPU links

INSIGHT

GPUs win because the work is independent: detector 1 on "I" doesn't wait for detector 2 on "love".

WATCH OUT

One-token-at-a-time generation is a one-row matrix, too small to keep the chip busy (Ch 18).



17 Memory: weights plus cache

Before speed comes fit. Weights and cache must sit in GPU memory.

How much memory does a model need?

COMES IN
The model file + live caches

→

WHO DOES THE WORK
GPU memory (80 GB on an H100)

→

HANDS ON
What must be read every step → bandwidth (Ch 18)

IN ONE LINE Weights are a fixed cost; the cache grows per user and per token.

Storage	Bytes per number	Llama 3.1 8B weights
16-bit (BF16)	2	8.03 B × 2 ≈ **16 GB**
8-bit	1	≈ 8 GB
4-bit	0.5	≈ 4–5 GB (some parts kept at higher precision)

Units: cache sizes use binary units (1 KB = 1,024 bytes, 1 GB = $1{,}024^3$ bytes, strictly KiB/GiB); weight sizes use decimal GB (10^9 bytes). All figures are rounded engineering estimates.

KV CACHE, LLAMA 3.1 8B AT 16-BIT

```
per token  = 128 KB
1 user,   8,192 tokens  =  1 GB
1 user, 131,072 tokens  = 16 GB   (= the weights)
10 users, 32,768 tokens = 40 GB   (2.5× the weights)
```

Decision matrix: which fix for which problem

Problem	Fix	What it changes	Cost
Weights don't fit	Quantisation	Fewer bytes per weight	Some quality loss, more at 4-bit
Cache too big per token	GQA (shared K/V) / MLA (compressed K/V)	Fewer numbers stored per token	Decided at training time
Cache too big overall	KV quantisation	Store cache in 8-bit	Small quality loss
Very long input	Sliding window / offload	Keep only recent tokens, or park old ones in CPU memory or storage	Forgets, or slower
Same prompt reused	Prefix caching	Reuse the cache across requests	Memory to hold it
Memory wasted in gaps	PagedAttention	Cache stored in small fixed-size blocks	Engine complexity

Memory arithmetic, and loading in 4-bit — Python

```
weights_gb = 8.03e9 * 2 / 1e9                      # ~16 GB at 16-bit
kv_per_token = 2 * 32 * 8 * 128 * 2                # 131,072 bytes
print(weights_gb, kv_per_token * 32768 * 10 / 2**30)   # 10 users x 32K: 40 GiB

from transformers import BitsAndBytesConfig        # pip install bitsandbytes
q = BitsAndBytesConfig(load_in_4bit=True)
model = AutoModelForCausalLM.from_pretrained(name, quantization_config=q)
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

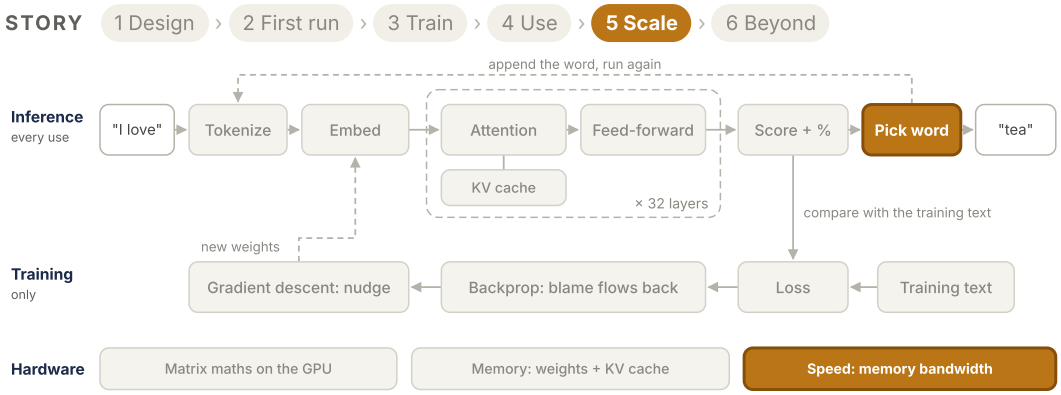
Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Bytes per weight	2 → 1 → 0.5	Fewer bytes to read → faster decoding	16 → 8 → ≈ 4–5 GB	Fits smaller GPUs and laptops	Quality loss grows at low bits
Parameters	8 B → 70 B → 405 B	∝ parameters	≈ 16 → 140 → 810 GB at 16-bit	Generally more capable	Several GPUs; slower and costlier

INSIGHT

Weights are a fixed cost. The cache is per user, per token. At scale the cache often decides how many users fit on a GPU.

WATCH OUT

Real inference memory = weights + KV cache + temporary working values + framework overhead. Training memory is far larger again (Ch 12: ≈ 16 bytes per weight before working values).



18 Compute vs memory: why answers type out word by word

An H100 can do ≈ 989 trillion operations a second. One Llama token needs ≈ 16 billion. So why isn't it instant?

What limits the speed of an answer?

COMES IN
Weights + cache in memory (Ch 17)

→

WHO DOES THE WORK
Memory bandwidth: ≈ 3.35 TB/s

→

HANDS ON
Tokens per second → you

IN ONE LINE Single-user decode speed is mostly limited by bytes moved per token.

ONE TOKEN, ONE USER, LLAMA 3.1 8B ON ONE H100

```
Maths:          16 billion ops   ÷ ~989 trillion/s  ≈ 0.016 ms
Weights to read: 16 GB           ÷ ~3.35 TB/s       ≈ 4.8 ms   ← the real wait
Ceiling ≈ 1 ÷ 4.8 ms ≈ 200 tokens/s
```

For one user, each new token needs **all the weights** read from memory, plus the cache, but only a little maths per number read. The calculators wait on memory.

WHAT THIS ESTIMATE IS, AND ISN'T

An **idealised ceiling**: one user, 16-bit weights, no batching, perfect use of bandwidth, cache traffic ignored. Real throughput depends on batching, quantisation, KV-cache size, kernel efficiency and the serving software. Use it to reason about *what limits speed*, not to predict a benchmark.

	Prefill (reading your prompt)	Decode (writing the answer)
Tokens per pass	All at once	One
Weights read	Once for the whole prompt	Once per token
Limited by	**Compute**	**Memory bandwidth**
You feel it as	Wait before the first word	Speed of the words

Batching fixes most of it: read the weights once and apply them to many users' next tokens together. The catch: each user brings a separate cache.

The speed ceiling in three lines — Python

```python
bytes_per_token = 16.06e9            # all weights, 16-bit
bandwidth = 3.35e12                  # H100 SXM, bytes per second
print(bandwidth / bytes_per_token)   # ~209 tokens/s ceiling for one user
```

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Users per batch	1 → 32	One weight read serves 32 tokens: far more total throughput	32 caches	Lower cost per token	Memory fills; each user may slow down
Memory bandwidth	—	Directly sets the decode ceiling	—	Faster typing-out	The expensive part of the chip

INSIGHT

Almost every inference trick (quantisation, GQA/MLA, batching, KV compression) is about **moving fewer bytes per token**, not doing less maths.

MYTH

Models type word by word for effect.

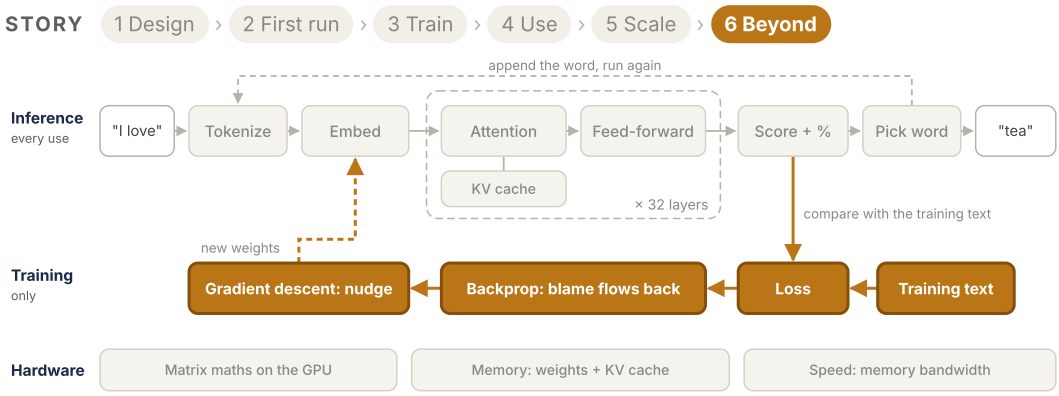
REALITY

Each word genuinely needs a full pass that reads every weight.

STAGE 6

Beyond pre-training: how today's large systems go further

A pre-trained model predicts text. Assistants that follow instructions, prefer good answers and use tools get three more layers: fine-tuning, reinforcement learning, and a harness of code around the model.



19 Fine-tuning: more training, on new text

Pre-training is expensive and done once. Fine-tuning reuses the trained weights and trains a little more on a smaller, targeted text.

How do you change a trained model's behaviour, and what does it cost?

COMES IN	WHO DOES THE WORK	HANDS ON
Trained weights (Ch 12) + new text	The same training loop (Ch 10–12)	Adjusted weights → use (Ch 13–15)

IN ONE LINE Fine-tuning is the same training loop on new text, starting from trained weights.

A café wants the model to recommend coffee. New text: **"I love coffee" × 3**. Same loss, same backprop, same update rule, learning rate 0.1. Only the starting point differs: trained weights instead of random ones.

Option 1: full fine-tuning (all 80 weights move)

Step	"I love" → tea	→ coffee	"I" → love / hate	Loss on the original text
0	66%	33%	73% / 26%	0.523
5	3%	97%	98% / 2%	1.392
10	2%	98%	99% / 1%	1.627
20	1%	99%	99% / 1%	1.895
40	0%	100%	100% / 0%	2.197

- Coffee jumps from 33% to 97% in just 5 steps.
- **Side effect:** "I" → hate falls from 26% to 0.2%, though we never asked for that. The new text never contains "hate", so training pushed it down. Loss on the original text rose from 0.523 to 2.197. This is **catastrophic forgetting**.

Option 2: LoRA (freeze the model, train a small add-on)

LoRA (low-rank adaptation) freezes every original weight and adds a small correction built from two thin matrices. We add one to the feed-forward write rows: a 3 × 1 column times a 1 × 4 row = **7 trainable numbers** instead of 80. The row starts at zero, so training begins from the unchanged model.

Step	"I love" → tea	→ coffee	"I" → love / hate	Loss on the original text
0	66%	33%	73% / 26%	0.523
5	65%	35%	74% / 26%	0.523
10	63%	37%	74% / 25%	0.523
20	54%	45%	77% / 23%	0.534
40	17%	83%	87% / 13%	0.774

- Coffee rises more slowly (83% after 40 steps), and the original text is damaged far less (loss 0.774 vs 2.197).
- Fewer trainable numbers means less to store and less to disturb. It's a trade-off: less change and less forgetting.

LoRA in a few lines — Python

```
A = rng.normal(0, 0.5, size=(3, 1))   # small random column
B = np.zeros((1, 4))                  # starts at zero: no change at step 0
W2_used = W2_frozen + A @ B           # only A and B are trained (7 numbers)

# Real models, with the peft library:
from peft import LoraConfig, get_peft_model
model = get_peft_model(model, LoraConfig(r=16, target_modules=["q_proj", "v_proj"]))
```

IN A REAL MODEL (LLAMA 3.1 8B)

Rank-16 LoRA on Llama 3.1 8B's query and value matrices trains ≈ 6.8 million numbers, about 0.08% of the model. Llama 3's own post-training used several rounds of supervised fine-tuning on curated examples, plus preference training (Ch 20).

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Full fine-tuning	80 → 8 billion trainable	Training compute ∝ tokens × 8 B	≈ 16 bytes per weight with Adam ≈ 128 GB+	Biggest possible change	Several GPUs; risk of forgetting
LoRA	7 → ≈ 7 million trainable	Backward pass still runs through the whole model	Frozen base at 2 bytes/weight (16 GB) + small adapter and its optimiser state	Fits on one GPU; adapters are megabytes and swappable	Smaller change; may underfit big shifts

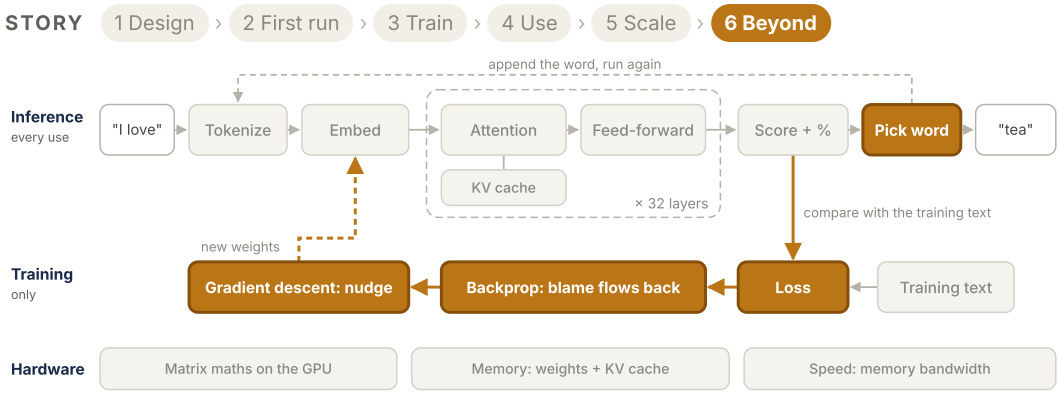
INSIGHT

Fine-tuning teaches **behaviour and format** well: tone, style, "always answer in JSON". For facts that change often, retrieval (putting text in the prompt) is usually cheaper and easier to update.

WATCH OUT

Fine-tuning data only shows what to do more of. Anything it leaves out may drift, as "hate" did here. Teams mix in some original-style data and test old behaviour after every run.

MYTH	REALITY
Fine-tuning adds a new knowledge module.	It nudges existing weights (or a small add-on) with the same loop as pre-training.



20 Reinforcement learning: rewarding answers, not copying text

Fine-tuning needs example answers. Reinforcement learning only needs a score for answers the model produced itself.

How do you train toward "better" when you can't write the perfect answer?

COMES IN	WHO DOES THE WORK	HANDS ON
Trained model + a reward rule	Reward, then the same backprop and nudge	A model that prefers rewarded answers → use

IN ONE LINE RL raises the share of rewarded answers; a penalty keeps the model close to where it started.

The loop changes in one place. Instead of comparing with a text, the model **tries** answers, a **reward** scores them, and the update raises the share of answers that scored above average.

#	What happens	Our numbers
1	The model generates answers (sampling, Ch 14)	4 tries after "I love": tea, coffee, tea, tea
2	A reward scores each one	café rule: coffee = 1, anything else = 0 → [0, 1, 0, 0]
3	**Advantage** = how much better than the group average	mean 0.25, spread 0.43 → [−0.58, 1.73, −0.58, −0.58]
4	Backprop + nudge: raise the share of above-average answers, lower the rest	coffee up, tea down

Step 3 is the group-relative scoring used by some reasoning-model training (e.g. GRPO, used for DeepSeek-R1). Classic RLHF instead trains a **reward model** on human comparisons ("answer A is better than B") and optimises against it.

Our run: maximise reward, with and without a "stay close" penalty

The penalty is the **KL divergence**: a number that grows as the new percentages drift from the original model's. Objective = reward − β × KL. β sets how much drifting costs.

Step	β = 0: coffee	Loss on original text	β = 0.5: coffee	KL	Loss on original text
0	33.3%	0.523	33.3%	0.000	0.523
10	99.4%	1.782	78.6%	0.433	0.702
25	99.8%	2.184	78.6%	0.434	0.702
50	99.9%	2.558	78.6%	0.433	0.702
100	100.0%	3.013	78.6%	0.433	0.702
200	100.0%	3.555	78.7%	0.433	0.702
300	100.0%	3.908	78.7%	0.433	0.703

- **No penalty (β = 0):** coffee hits 100% and the model's fit to its original text collapses (loss 0.523 → 3.908). It maximised the reward and gave up everything else.
- **With penalty (β = 0.5):** coffee settles at **78.7%**. Theory predicts the balance point exactly: new share ∝ old share × e^(reward ÷ β), which gives **78.7%**. Our training landed on it. (β = 1 would give 57.6%.)

The RL step, and the closed-form check — Python

```
rewards = np.array([0, 1, 0, 0])                # one score per sampled answer
adv = (rewards - rewards.mean()) / rewards.std() # [-0.58, 1.73, -0.58, -0.58]
# update: weights += lr * sum(adv_i * gradient of log p(answer_i))

p_ref = np.array([.664, .333])                  # tea, coffee before RL
r, beta = np.array([0, 1]), 0.5
p_best = p_ref * np.exp(r / beta); p_best /= p_best.sum()   # coffee ~ 0.787
```

IN A REAL MODEL (LLAMA 3.1 8B)

Published recipes vary. InstructGPT-style RLHF used a learned reward model, PPO and a KL penalty. Llama 3 used Direct Preference Optimisation (DPO), which learns from preference pairs without a separate RL loop, over six rounds. Reasoning models are often trained with **checkable rewards**: did the maths answer match, did the code pass its tests?

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Samples per prompt	4 → many per prompt, millions of prompts	Generating answers is inference: slow, token by token, and often the biggest cost of RL	Policy, reference model and sometimes a reward model in memory together	Learns from its own attempts; needs no perfect example answers	Expensive; unstable without care
Answer length	1 word → thousands of reasoning tokens	Longer answers cost more per sample	More KV cache per sample	Reasoning models learn to think longer when it raises reward	Longer, costlier answers at use time too

WATCH OUT: REWARD HACKING

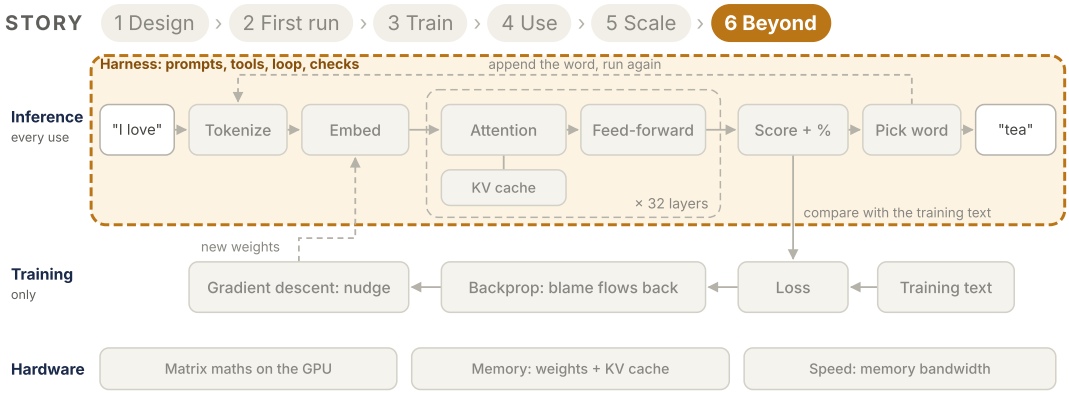
The model maximises the **score**, not your intention. With β = 0 ours abandoned everything for the reward. Learned reward models can be fooled by answers that look good but aren't, which is one reason checkable rewards and KL penalties are used.

MYTH

RL teaches the model new facts.

REALITY

Mostly it reshapes which of the model's existing behaviours become likely. Here it moved shares between tea and coffee; it couldn't invent a drink it never learned.



21 The harness: the code wrapped around the model

What you use in a product is rarely the bare model. It's a loop of plain code that builds the prompt, calls tools, checks results and calls the model again.

What does the system around the model do, and what's the arithmetic of it?

COMES IN	→	WHO DOES THE WORK	→	HANDS ON
A trained model		Harness: ordinary code		Answers, actions, results → you

IN ONE LINE Most of what an AI product does happens in plain code around the model.

The model only ever does one thing: given tokens, output percentages for the next token. Everything else is code:

Harness part	What it does	How it reaches the model
System prompt	Instructions, persona, rules	Tokens placed before your message
Retrieval (RAG)	Finds relevant documents	Pasted into the prompt as tokens
Tools	Search, calculator, code runner, database	Model writes a structured request; code runs it; the result is appended as tokens
Loop / agent	Repeats think → act → observe until done	Each turn re-sends the growing context
Checks	Tests, validators, filters, retries	Code decides whether to accept or retry
Routing	Picks a small or large model per request	Different model, same interface
Chat interface	Keeps the history, applies the chat template, streams the reply	The whole conversation, re-sent as tokens each turn

The chat interface: the harness you use every day

A chat window feels like a conversation. The model sees something simpler: **one long token stream**, rebuilt and re-sent on every turn.

#	What happens	Example
1	You type a message	"What goes with tea?"
2	The app adds the system prompt and **every earlier message**	system + turn 1 + reply 1 + … + your new message
3	A **chat template** joins them into one text, with special tokens marking who said what	see below
4	The model predicts the reply token by token; the app **streams** each token to your screen as it arrives	"Biscuits…"
5	It stops at the end-of-turn token; the app saves the reply into the history	ready for the next turn

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>
What goes with tea?<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

Llama 3.1's chat format. The model continues from the last line; the special tokens were learned in fine-tuning (Ch 19).

What it feels like	What actually happens
The model remembers our chat	The app re-sends the whole history every turn. The model itself keeps nothing between calls
It types as it thinks	Streaming: each token is shown as soon as it's picked (Ch 14)
A long chat "forgets" the start	History exceeded the context window, so the app dropped or summarised old turns
It remembers me across chats	A memory feature: the app stores notes and pastes them into the prompt

CHAT ARITHMETIC, LLAMA 3.1 8B

```
Each turn (question + reply) ≈ 500 tokens
Turn 20 re-sends ≈ 10,000 tokens   → ≈ 1.2 GB of KV cache
Context limit 131,072 tokens        → full after ≈ 260 turns; then the app must trim
```

Without prefix caching, every turn re-reads the whole history in prefill, which is why long chats get slower and costlier.

The arithmetic that shapes harness design

Question	Calculation	Result
Long multi-step tasks	Each step succeeds 95% of the time; all must succeed: 0.95 × 0.95 × …	5 steps: 77% · 10 steps: **60%** · 20 steps: 36%
Same, with better steps	0.99 per step	10 steps: 90% · 20 steps: 82%
Majority vote	Ask 5 times, take the majority; each answer right 60% of the time	5 votes: **68%** · 9 votes: 73%
Best of n with a checker	Each try right 30%; tests can tell which tries pass	1 try: 30% · 5 tries: **83%** · 10 tries: 97%
Cost of a 10-turn loop	Each turn adds 2,000 tokens and re-sends everything so far: 2,000 × (1 + 2 + … + 10)	**110,000** input tokens, not 20,000 (prefix caching cuts the re-reading cost)

- Reliability **multiplies** across steps, which is why harnesses add checks and retries after risky steps.
- Voting and best-of-n trade **more tokens** (cost, time) for accuracy. This is one form of "test-time compute". Majority voting over reasoning paths is called **self-consistency**.
- The voting and retry figures assume independent attempts. Real attempts make correlated mistakes, so real gains are smaller.

Chat template and a minimal tool-using loop — Python

```
text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)  # one
token stream

messages = [{"role": "system", "content": "Use tools when needed."},
            {"role": "user",   "content": "What is 1234 * 5678?"}]
while True:
    reply = model(messages)                          # the model: next-token prediction
    if reply.tool_call:                              # it asked for a tool, as text/JSON
        result = run_tool(reply.tool_call)           # plain code, e.g. a calculator
        messages += [reply, {"role": "tool", "content": str(result)}]
        continue                                     # call the model again
    break                                            # final answer
```

IN A REAL MODEL (LLAMA 3.1 8B)

Chat assistants, coding agents and research tools are a trained model (Ch 1–20) inside a harness like this. Speed and cost follow Chapters 15–18: every loop re-reads context, so the KV cache, prefix caching and batching matter even more for agents.

SCALE IT UP: WHAT GROWS, AND WHAT IT COSTS

Knob	Toy → Llama 8B	Compute	Memory	Good	Bad
Loop turns	1 → dozens	Each turn is a full model call	Context and KV cache grow every turn	Can finish multi-step tasks	Cost grows faster than the number of turns; errors compound
Samples per question	1 → n (voting, best-of-n)	n × the tokens	n × the cache, or n × the time	Higher accuracy where answers can be checked or compared	n × the cost

INSIGHT

Much of the practical gain in today's AI products comes from the harness: retrieval, tools, checking and retries. These are engineering choices you can reason about with the arithmetic above.

MYTH

The model browses, runs code and remembers you.

REALITY

The model outputs text. The harness does the browsing, running and remembering, then feeds the results back in as tokens.

APPENDIX

The whole story on one page

Every number from the tiny model, in order.

Stage	Step	Result	Ch
Design	Training text	4 sentences → targets: after "I love", tea 67% / coffee 33%	1
Design	Vocabulary	6 words, ids 0–5	2
Design	Shape	4 slots, 1 layer, 1 head, 3 detectors → 80 weights	3
First run	Random weights → "I love"	tea 18%, coffee 18%, rain 16%: noise	4–9
Train	Loss	1.864 (best possible 0.520)	10
Train	One gradient, two ways	−0.0123 (chain rule) = −0.0123 (nudge)	11
Train	500 steps, learning rate 0.1	loss 0.523; tea–coffee similarity 0.00 → 0.92	12
Use	Trained "I love"	tea 66%, coffee 33%, rain 0.2%	13
Use	"I love tea" → ?	rain 95%: confident, never trained	14
Use	KV cache	Same answer, only 1 new token computed	15
Beyond	Full fine-tune on "I love coffee"	coffee 97% in 5 steps; "I" → hate falls 26% → 0.2% (forgetting)	19
Beyond	LoRA, 7 trainable numbers	coffee 83%, much less forgetting	19
Beyond	RL, reward coffee, β = 0.5	coffee 78.7%, as theory predicts	20

Scaling knobs at a glance

Knob	Compute grows	Memory grows	You feel it as	Ch
Parameters	∝ parameters	∝ parameters × bytes	Which GPU, cost per token	3, 17
Width	≈ width²	≈ width²	Model size class	3
Context length	Attention scores ≈ length²	KV cache ∝ length	Long prompts slow and pricey	6, 15
Concurrent users	∝ users (batched)	KV cache ∝ users	Users per GPU	18
Output length	∝ tokens, one at a time	KV cache ∝ tokens	Wait time, output price	14
Bytes per number	Less to read → faster decode	∝ bytes	Fits smaller hardware	17
Training tokens	≈ 6 × parameters × tokens	Streamed, not on GPU	Training bill	1, 12

Glossary

- **Bandwidth**: bytes per second memory can deliver
- **Batch**: several requests processed together
- **Causal mask**: tokens may only look backwards
- **Context**: all tokens the model can see
- **Cross-entropy**: the loss; surprise at the actual next token
- **Decode**: writing, one token per pass
- **Gradient**: how loss changes as one weight moves
- **GQA**: query heads sharing K/V to shrink the cache
- **Head**: one independent attention calculation
- **Inference**: using fixed weights to answer
- **Learning rate**: size of each nudge
- **Logit**: raw score before softmax
- **MLA**: compressing K/V into a smaller row

- **Parameter**: one learned number
- **Prefill**: reading the prompt in one pass
- **Quantisation**: fewer bits per number
- **Residual shortcut**: adding a change instead of replacing
- **Top-p**: sample only from the top words covering p%
- **Chat template**: format that joins messages into one token stream
- **Fine-tuning**: more training on targeted text
- **Harness**: code around the model (chat, tools, loops)
- **KL penalty**: cost for drifting from the original model
- **LoRA**: train a small add-on, freeze the model
- **Reward hacking**: maximising the score, not the intent

Sources and method

- **Tiny model:** every number computed in Python (NumPy): random start (seed 1, spread 0.5, rounded to 1 decimal), plain gradient descent, learning rate 0.1, 500 full-batch steps on the 8 predictions. Gradients computed numerically and cross-checked with the chain rule (Ch 11). Displayed values are rounded.
- **Llama 3.1 8B architecture** (4,096 width, 32 layers, 32 query / 8 key-value heads, 14,336 feed-forward, 128,256 vocabulary, untied output table, initializer_range 0.02, 131,072 positions): model config.json, huggingface.co/meta-llama/Llama-3.1-8B
- **Training:** ≈ 15 trillion tokens, 1.46 M H100-80GB GPU-hours for 8B: Meta Llama 3.1 model card, huggingface.co/meta-llama/Llama-3.1-8B. Peak learning rate 3×10^{-4}, AdamW, GQA, SwiGLU, RoPE, tokenizer (100K tiktoken + 28K), 3.94 characters per token: "The Llama 3 Herd of Models" (Meta, 2024), arxiv.org/abs/2407.21783
- **H100 SXM:** 80 GB HBM3, ≈ 3.35 TB/s, ≈ 989 dense BF16 TFLOPS: NVIDIA H100 datasheet, nvidia.com/en-us/data-center/h100
- **16 bytes per parameter** for mixed-precision Adam: ZeRO, Rajbhandari et al. 2020, arxiv.org/abs/1910.02054. **6 × parameters × tokens**: Kaplan et al. 2020, arxiv.org/abs/2001.08361
- **100 tokens ≈ 75 words** (English rule of thumb): OpenAI Help Center, help.openai.com/en/articles/4936856
- **Post-training:** RLHF with reward model, PPO and KL penalty: Ouyang et al. 2022 (InstructGPT), arxiv.org/abs/2203.02155. Llama 3: six rounds of SFT + rejection sampling + DPO, PPO explored but not used: arxiv.org/abs/2407.21783. DPO: Rafailov et al. 2023, arxiv.org/abs/2305.18290. Rule-based rewards and group-relative advantages (GRPO): DeepSeek-R1, arxiv.org/abs/2501.12948. KL-regularised optimum p ∝ p_ref · e^(r/β): standard result, e.g. in the DPO paper.
- **LoRA:** Hu et al. 2021, arxiv.org/abs/2106.09685 (A random, B zero at start). **Self-consistency** (majority vote over reasoning paths): Wang et al. 2022, arxiv.org/abs/2203.11171. Llama 3.1 chat format: Meta model card.